Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3558005

PDF Download
3558005.pdf
28 January 2026
Total Citations: 98
Total Downloads: 3821

Published: 22 March 2023
Online AM: 23 August 2022
Accepted: 15 August 2022
Revised: 22 June 2022
Received: 03 November 2021

Citation in BibTeX format

RESEARCH-ARTICLE

Personalized Federated Learning on Non-IID Data via Group-based Meta-learning

LEI YANG, South China University of Technology, Guangzhou, Guangdong, China

JIAMING HUANG, South China University of Technology, Guangzhou, Guangdong, China

WANYU LIN, The Hong Kong Polytechnic University, Hong Kong, Hong Kong, Hong Kong

JIANNONG CAO, The Hong Kong Polytechnic University, Hong Kong, Hong Kong, Hong Kong

Open Access Support provided by:

South China University of Technology

The Hong Kong Polytechnic University

Personalized Federated Learning on Non-IID Data via Group-based Meta-learning

LEI YANG and JIAMING HUANG, South China University of Technology
WANYU LIN and JIANNONG CAO, The Hong Kong Polytechnic University

Personalized federated learning (PFL) has emerged as a paradigm to provide a personalized model that can fit the local data distribution of each client. One natural choice for PFL is to leverage the fast adaptation capability of meta-learning, where it first obtains a single global model, and each client achieves a personalized model by fine-tuning the global one with its local data. However, existing meta-learning-based approaches implicitly assume that the data distribution among different clients is similar, which may not be applicable due to the property of data heterogeneity in federated learning. In this work, we propose a Group-based Federated Meta-Learning framework, called *G-FML*, which adaptively divides the clients into groups based on the similarity of their data distribution, and the personalized models are obtained with meta-learning within each group. In particular, we develop a simple yet effective grouping mechanism to adaptively partition the clients into multiple groups. Our mechanism ensures that each group is formed by the clients with similar data distribution such that the group-wise meta-model can achieve "personalization" at large. By doing so, our framework can be generalized to a highly heterogeneous environment. We evaluate the effectiveness of our proposed *G-FML* framework on three heterogeneous benchmarking datasets. The experimental results show that our framework improves the model accuracy by up to 13.15% relative to the state-of-the-art federated meta-learning.

CCS Concepts: • **Security and privacy**; • **Computing methodologies** → **Machine learning**; **Artificial intelligence;**;

Additional Key Words and Phrases: Federated learning, neural networks, clustering methods, meta learning

ACM Reference format:
Lei Yang, Jiaming Huang, Wanyu Lin, and Jiannong Cao. 2023. Personalized Federated Learning on Non-IID Data via Group-based Meta-learning. *ACM Trans. Knowl. Discov. Data.* 17, 4, Article 49 (March 2023), 20 pages.
https://doi.org/10.1145/3558005

This work is supported in part by the National Natural Science Foundation of China under Grant 61972161, in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2022A1515010374, in part by Hong Kong RGC General Research Fund under Grant PolyU 15220020 and PolyU 15204921, in part by Hong Kong RGC Research Impact Fund under Grant R5060-19, and in part by the Hong Kong RGC Theme-based Research Scheme (TRS) under Grant T41-603/20-R.
Authors' addresses: L. Yang and J. Huang, South China University of Technology, China; emails: sely@scut.edu.cn, 201921043977@mail.scut.edu.cn; W. Lin (corresponding author) and J. Cao, The Hong Kong Polytechnic University, China; emails: {wanylin, csjcao}@comp.polyu.edu.hk.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Association for Computing Machinery.
1556-4681/2023/03-ART49 $15.00
https://doi.org/10.1145/3558005

1 INTRODUCTION

It has been widely recognized that neural network models lead to significant gains in their results with a large amount of training data. In many applications, these data are generated and stored by end-users. Due to privacy concerns, it is not practical to collect all the data from the users to a central server for centralized training. Therefore, it is necessary and essential to train the neural network models in a distributed manner, with no need to collect the data in a central node for training. For this purpose, **Federated Learning** (**FL**) [1] was proposed that enables a large number of edge devices to collaboratively learn a shared model without data sharing. In FL, as the data source and computing nodes are end-user devices, the data distribution on each device is usually different, commonly referred to as Non-IID (a.k.a. Non-Independent and Identically Distributed) in the literature [1–3]. For example, in a movie recommendation system [4], a predictive model is trained over all the data across various users. Typically, the rating of each user is dominated by his/her preference. A single global model is difficult to adapt to each user due to the data heterogeneity across users. Figure 1 gives a visual illustration of this situation when clients are partitioned into three groups due to various preferences. To this end, **personalized federated learning** (**PFL**) [5–9] was proposed to provide a personalized model for each user while enjoying all the benefits of the vanilla FL architecture. One representative approach for the personalized FL framework is Federated Meta-Learning, which provides personalized models for each user by leveraging the personalization capability of meta-learning. Specifically, federated meta-learning approaches [6, 10, 11] utilize meta-learning methods, such as MAML [12] or Reptile [13], in which each client can learn its personalized model by several gradient descents.

Despite the great success achieved by federated meta-learning [6, 14–16], most of these works heavily rely on a single global meta-model to achieve customized models for each client. In addition, federated meta-learning approaches usually assume that the local data distribution should not deviate too much statistically from the global data distribution to guarantee their convergence. However, this assumption does not always hold in real-world applications. As shown in Figure 1, if the data distributions among clients are significantly diverse, one global meta-model may be insufficient to fit each data distribution. Thus, relying on the guidance of only one global meta-model may degrade the overall performance of personalized models.

Regarding the above insights, in this work, we propose a novel learning framework, namely *G-FML*, which learns multiple group-wise meta-models to fit different data distributions simultaneously. *G-FML* aims at learning accurate and personalized models despite the great dispersion of the data distribution among various clients. In particular, to learn multiple group-wise meta-models, we first develop an efficient grouping algorithm, and within each group, our framework learns a group-wise meta-model. As our group mechanism tends to characterize various data distributions across the clients, the main challenge of our problem is *how to quantify the data distribution of the client*. For this purpose, we propose utilizing a deep autoencoder to learn the low-dimensional feature representations of the local data in each client. As these low-dimensional representations are the most salient features of the data, we aggregate them as a whole to represent the data distribution of each client. For simplicity, we refer to the aggregated representation as the task representation. In addition, due to the limited computation capacity in end-user devices, we perform local updates using a small batch of the local data. In this way, the task representations are different in each communication round, which may affect the grouping results. Therefore, our grouping strategy is naturally adaptive in the sense that it can efficiently perform re-grouping clients every communication round.

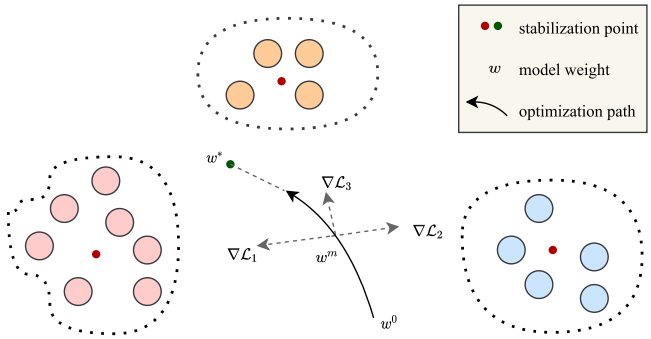

Fig. 1. An example of challenges for FL with Non-IID data. Obviously, a single global model/meta-model is difficult to adapt to all data distributions.

The main contributions of this work are as follows:

(1) We identify and empirically demonstrate the limitation of federated meta-learning approaches that rely on a single meta-model in an environment with highly skewed Non-IID data.
(2) We propose a framework and an efficient grouping algorithm to address this limitation. Specifically, the proposed framework generates a set of global meta-models and accurately assigns them to the clients.
(3) Experimental results on both synthetic and real datasets show that our *G-FML* is more effective than state-of-the-art federated meta-learning in terms of model accuracy and convergence speed.

The remainder of this work is organized as follows. We first discuss the related work about PFL in Section 2. In Section 3, we define our problem formally and list the major notations in this work. Section 4 elaborates our proposed *G-FML* framework and methods. Section 5 presents experimental settings and shows the evaluations of our framework on federated datasets. Finally, Section 6 presents the conclusion of this work and outlines future directions.

2 RELATED WORK

FL [1, 17–20] enables clients to collaboratively train machine learning models in a privacy-preserving manner. Since the data is stored in the client and the data among clients usually comes from a different but not completely unrelated distribution, this leads to the statistical challenges in FL such as Non-IID data and unbalanced data. To overcome these statistical challenges, PFL [21, 22] is a new paradigm of FL that has attracted much attention recently.

Tan et al. [22] categorize PFL methods into data-based and model-based approaches. Data-based approaches [2, 23–26] aim at smoothing the statistical heterogeneity among clients' dataset to mitigate the problem of weight divergence during the FL training process. Model-based approaches aim at enabling FL models to adapt to the diverse data distributions among clients and can further be classified into three methods: single-model PFL approaches [6, 8, 11, 14], multi-model PFL approaches [27–30] and N-model PFL approaches [5, 7, 31, 32]. Our work is mainly related to the single-model and multi-model PFL approaches. We summarize the related work in the following subsections.

2.1 Multi-Model PFL Approaches

One category of multi-model PFL approaches is **clustered federated learning** (**CFL**). CFL assumes that the data distributions among clients are significantly different, and the goal of CFL is

to divide clients into clusters of similar distributions and each cluster maintains a single model. Some of these works focus on using iterative schemes for clustering. In [27], the authors develop a recursive bi-partitioning algorithm based on cosine similarity to divide clients into clusters. It performs FedAvg [1] and bi-partitioning clustering in turn until convergence. Ghosh et al. [28] propose an iterative optimization algorithm for CFL with alternately cluster identity estimation and model optimization in each communication round. In [33], the authors propose a multi-center aggregation FL scheme that learns multiple global models to tackle the Non-IID challenge. They present the **federated stochastic expectation maximization** (**FeSEM**) to solve the optimization of the proposed objective function. Some other works reduce clustering to a single step. Briggs et al. [29] propose a hierarchical clustering framework for FL, which separates clusters of clients by the similarity of their local updates to the global joint model and each cluster is trained independently. Different from [29], Duan et al. [30] divide aggregation into intra-group aggregation and inter-group aggregation, where a group model will be aggregated after intra-group aggregation.

The CFL often incurs substantial communication and computation costs, which may limit its feasibility in practical applications [22]. In order to reduce communication costs, our grouping strategy uses the autoencoder to reduce the amount of data in the transmission process. For computation efficiency, we perform the cluster assignment by a single step based on cosine similarity, and we remove the cluster center update step.

2.2 Single-model PFL Approaches

For single-model PFL approaches, a global FL model or representation is trained and adapted to each client on local data. One category of single-model PFL approaches is using parameter decoupling [7, 14, 34, 35], which decouples the private parameters from the global model parameters. Arivazhagan et al. [14] propose FedPer, in which a neural network is divided into base and personalization layers, and the base layers are trained centrally, while the personalization layers are trained locally. In [34], the authors train a document classification model in FL, in which the private user embedding is kept on the device and the character embedding is trained for sharing among users.

Another category of single-model PFL approaches is federated meta-learning, which adopts meta-learning methods [36, 37] into FL for personalization. Jiang et al. [11] explore the connection between MAML and FL. They treat the global meta-model of MAML as the global model of FL and treat the task as the local model of the client. They also show that existing optimization-based meta-learning algorithms(e.g., MAML or Reptile) can be integrated into FL for personalization. In [6], the authors propose Per-FedAvg, a personalized variant of FedAvg based on MAML, to customize personalized models by training a good initial global model on the local data of a client. In addition, they characterize how the convergence of Per-FedAvg is affected by the closeness of distributions of user data, measured by distribution distances, such as Total Variation or 1-Wasserstein. From a theoretic point of view, Lin et al. [38] study the convergence of the federated meta-learning and investigate the impact of the node similarity. In contrast to MAML-type methods, Khodak et al. [39] propose ARUBA, an online learning algorithm for adaptive meta-learning, and shows that it improves the performance of FedAvg.

However, these works focus on seeking a global meta-model or representations shared among clients, which is not suitable for the situation where the local data distribution of clients is excessively different from the global data distribution [36, 40, 41]. In this work, we mitigate this problem by providing multiple global meta-models.

3 PROBLEM DEFINITION

In this section, we formalize the personalized FL problem. Table 1 presents the mathematical notations and associated descriptions.

Table 1. Mathematical Notations in This Work

Symbol	Description
i	index of each client;
k	index of each group;
N	the number of clients that participated in the whole training process;
$\mathcal{S}$	subset of clients;
K	total number of groups;
C_k	cluster center on the kth group;
G_k	set of client indexes on the kth group;
H, h	representation on data using autoencoder;
ϕ_k	meta-model on the kth group;
$\mathcal{D}_i$	dataset on client i;
θ_i	personalized model of client i;
$\mathcal{L}_i$	local empirical risk of client i.

In FL, there are N clients collaboratively training a shared global model θ. Let i denotes the index of the client. Each client i holds a local dataset $\mathcal{D}_i$ with arbitrary size. The loss of model θ for a data sample d drawn from $\mathcal{D}_i$ is given by $\ell_i(\theta; d)$. We denote $\mathcal{L}_i([\theta]; \mathcal{D}_i) = \mathbb{E}_{d \in \mathcal{D}_i}[\ell_i(\theta; d)]$ as the empirical risk of model θ with respect to the local dataset $\mathcal{D}_i$. The goal of FL is to solve an empirical risk minimization problem as follows:

$$\min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}_i([\theta]; \mathcal{D}_i), \quad i \in [N], \tag{1}$$

where Θ denotes the parameter space.

Suppose the ith client IID samples data from the distribution P_i. Then the global data distribution is a mixture of all local data distributions $P = \sum_{i=1}^{N} w_i P_i$, where the weight w_i of each client depends on local data size. In the IID assumption, the data among all clients are IID partitioned, that is, $P_i = P$ for all $i \in [N]$. However, the IID assumption does not typically hold in real-world applications [19]. Therefore, if data tend to deviate from being identically distributed P, that is $P_i \neq P_j$ for different clients i and j. We refer to this as not independent and identically distributed (Non-IID).

Due to the presence of Non-IID data, the global model θ could not achieve the minimum of the empirical risk $\mathcal{L}(\cdot)$ over all the clients. This has stimulated the exploration of personalized solutions. Most of the previous work has focused on incorporating some amount of personalized information (learned from a client's local data) into a global shared model [6, 16]. However, as we discussed in Section 1, one global model may become inefficient in a highly heterogeneous environment. We tackle this problem by sharing multiple global models among clients. More specifically, we use multiple group-wise meta-models.

Formally, we consider a set of global meta-models with a size of K, where each global meta-model is shared among a group of clients and each group has only one global model. These global meta-models are regarded as good model parameters for the model initialization. Thus, the client could choose a global meta-model that most closely fits its local data distribution to perform personalization, which motivates the following re-written global objective:

$$\min_{\phi_{g_i} \in \Phi, \theta_i \in \Theta} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}_i([\phi_{g_i}, \theta_i]; \mathcal{D}_i), \quad i \in [N]; g_i \in [K], \tag{2}$$

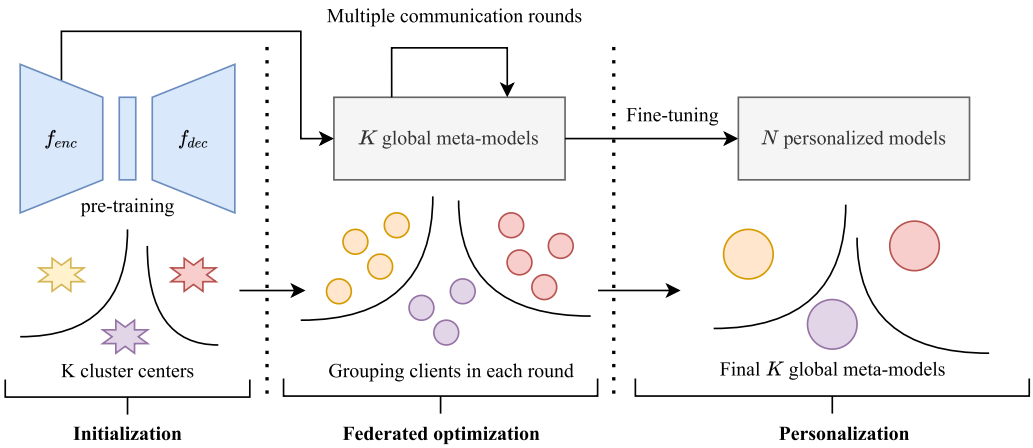

Fig. 2. Overview of *G-FML*.

where the g_i is the identify of the global meta-model to the i-client, Φ denotes the parameter space of global models and G_k is the set of client indexes on the kth group.

4 OUR FRAMEWORK: GROUP-BASED FEDERATED META-LEARNING

The crux of our framework, *G-FML*, is to partition the clients with similar data distribution into the same group for personalization meta-model learning through a grouping mechanism. Figure 2 depicts the framework of *G-FML*. Precisely, the entire training process of our framework consists of three stages: (1) the initialization stage; (2) the federated optimization stage; and (3) the personalization stage. In what follows, we will first conceptually discuss the intuition of our design in each stage, and then we will provide the detailed algorithms in our framework.

4.1 Overview of *G-FML*

The initialization stage. To capture the relationship between clients thoroughly, we first use an autoencoder to learn the task representation of each client and then divide clients into groups according to these task representations. Specifically, each client downloads the autoencoder architecture from the server. First, the autoencoder works to extract the low-dimensional vector from the local data, namely by *task representations*. Then the client transmits the task representations to the server, using the input of these representations to execute the K-Means clustering algorithm. As a result, we can obtain K cluster centers. These cluster centers serve to identify groups in the Federated Optimization stage. In addition, the meta-model for each group has the same model initialization.

The federated optimization stage. In this stage, the goal of *G-FML* is to optimize the group-wise meta-models. Without loss of generality, this stage follows the standard process of FL. Concretely, each client first downloads the corresponding group-wise meta-model from the server and optimizes the group-wise meta-model with its local data. In addition, we use the pre-trained encoder to obtain the task representation of training data. Finally, the client transmits the updated group-wise meta-model and the task representation to the server. The server completes the grouping process by measuring and comparing the similarity of the task representations with the K cluster centers. After that, the server performs model aggregation on the meta-models in each group to obtain the new global meta-models for the next communication round. We repeat this process until some terminating criterion is fulfilled, i.e., the limited communication rounds.

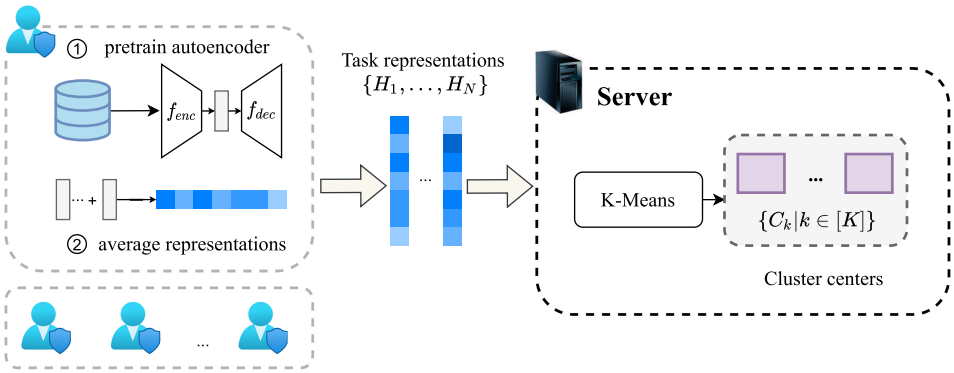

Fig. 3. Overview of the **Initialization** stage. ① All clients first train an autoencoder with their local data. ② Then clients compute the task representations using the encoder based on Equation (4). Finally, the server runs the K-Means algorithm with the task representations to obtain the cluster centers; these cluster centers will be used in the second stage.

The personalization stage. We follow the conventional design of using meta-learning for personalization in this stage. Specifically, we perform one-step gradient descent on the corresponding group-wise meta-model for each client. By doing so, each client can achieve a personalized model accordingly.

4.2 The Initialization Stage

Before we start the **Federated Optimization** stage, it would be necessary to perform certain useful initialization, including training the local autoencoder and obtaining the K cluster centers. Figure 3 describes the entire process of the Initialization stage. To learn a high-quality representation from the local data, we utilize an efficient latent embedding method, namely autoencoder, to map the data into a low-dimensional latent space. Each client trains the autoencoder in its data $\mathcal{D}$. Both the encoder f^i_{enc} and the decoder f^i_{dec} consist of two fully connected layers with ReLU activation. The reconstruction loss between the sample x and the autoencoder output $f^i_{dec}(f^i_{enc}(x))$ is as follows:

$$\mathcal{L}_i = \sum_{x \in \mathcal{D}_i} \left\| x - f^i_{dec}\left(f^i_{enc}(x)\right) \right\|_2^2, \tag{3}$$

where $f^i_{enc}(\cdot)$ and $f^i_{dec}(\cdot)$ are the encoder and the decoder of client i, respectively.

However, the autoencoder focuses on learning effective feature representations over samples, i.e., it learns a mapping of each sample rather than the entire dataset. With the background of sentence embedding methods in NLP [42, 43], many researchers are investigating how to perform sentence representation learning to obtain higher quality sentence embedding. A simple yet effective way to obtain sentence embedding is to combine the word embedding of all words in a sentence. Inspired by this, we aggregate representations of all samples to reflect the data distribution in client i. We can obtain representations of all samples through the encoder $f^i_{enc}(\cdot)$. Therefore, the representation of data distribution is an averaging over representations of all samples:

$$H_i = \frac{1}{|\mathcal{D}_i|} \sum_{x \in \mathcal{D}_i} f^i_{enc}(x), \tag{4}$$

where $H_i \in \mathbb{R}^d$ is the representation of data distribution in client i and $|\mathcal{D}_i|$ is the number of samples in $\mathcal{D}_i$.

Given the representation set of $\{H_i\}$, we utilize the K-Means clustering algorithm to obtain K cluster centers $C_1, \ldots, C_K$, which will serve in the **Federated Optimization** stage. After that, each cluster center corresponds to a group-wise meta-model with the same initialization. Finally, we present the entire process in Algorithm 1.

ALGORITHM 1: The process of initialization

Input: Number of groups K, dataset $\mathcal{D}_i$ of client
Output: List of cluster centers $\{C_1, \ldots, C_K\}$, list of encoders $\{f_{enc}^1(\cdot), \ldots, f_{enc}^N(\cdot)\}$
for each client i **in parallel do**
  Obtain the encoder: f_{enc}^i = **Autoencoder($\mathcal{D}_i$)**;
  Compute the task representation H_i of data $\mathcal{D}_i$ using f_{enc}^i based on Equation (8);
  Client i sends H_i back to server;
end
Run K-Means algorithm on $\{H_i\}$ to obtain the cluster centers $\{C_1, \ldots, C_K\}$;
Function Autoencoder($i, \mathcal{D}_i$)**:**
  Minimize the empirical loss $\mathbb{E}_{x \in \mathcal{D}_i} \mathcal{L}(x, f_{dec}(f_{enc}(x)))$ on the dataset $\mathcal{D}_i$ of client i;
  return f_{enc}^i
End Function

4.3 The Federated Optimization Stage

After the **Initialization** stage, we obtain the K cluster centers and the encoder $f_{enc}^i(\cdot)$ of client i. As shown in Figure 4, each client uses the pre-trained encoder f_{enc} to compress the training data into a task representation h, and the server performs grouping (based on task representation) and model aggregation (based on updated meta-models).

Specifically, we utilize the MAML algorithm to optimize the group-wise meta-model ϕ_k in a distributed manner. The key idea of MAML is to look for a meta-model such that it performs one or a few steps of gradient descent to achieve the best performance on the new task. Let $\mathcal{D}_i^{train}$ and $\mathcal{D}_i^{test}$ be the two disjoint set of $\mathcal{D}_i$, where $\mathcal{D}_i^{train}$ is used for meta-training and $\mathcal{D}_i^{test}$ is used for meta-testing. Assume each client uses one-step gradient descent to update the group-wise meta-model ϕ_k. We can rewrite the optimization problem in Equation (2) to:

$$\begin{aligned} &\min_{\phi_k} \sum_{i \in G_k} \mathcal{L}_i \left([\phi_k, \theta_i]; \mathcal{D}_i^{train''} \right), \\ &\text{s.t. } \theta_i = \phi_k - \alpha \nabla \mathcal{L}_i \left(\mathcal{D}_i^{train'} \right), \quad i \in [N]; k \in [K] \end{aligned} \tag{5}$$

where G_k denotes the set of client indexes on group k, $\mathcal{D}_i^{train'}$ and $\mathcal{D}_i^{train''}$ are the batch data with batch size $\mathcal{B}$ randomly sampled from $\mathcal{D}_i^{train}$, and α is the learning rate. Using batch data instead of the entire dataset to make gradient descent can reduce the computation cost for the client [6].

More specifically, the training process follows the standard FL process. Denote R as the number of communication rounds and denote $\mathcal{S}$ as the subset of randomly sampled clients. $|\mathcal{S}|$ denotes the number of selected clients per round $r \in R$. The group-wise meta-models ϕ_k are sent down to all clients in $\mathcal{S}$ based on the grouping result $G_1, \ldots, G_k$. Each client updates the group-wise meta-model by performing multiple epochs T on its data. As outlined in Algorithm 2, there are three main steps, i.e., update step, grouping step, and aggregation step.

Update step: We perform this step on the client-side. In epoch $t \in T$, each client first receives corresponding group-wise meta-model ϕ_k^t from the server and updates local model θ_i^t using a

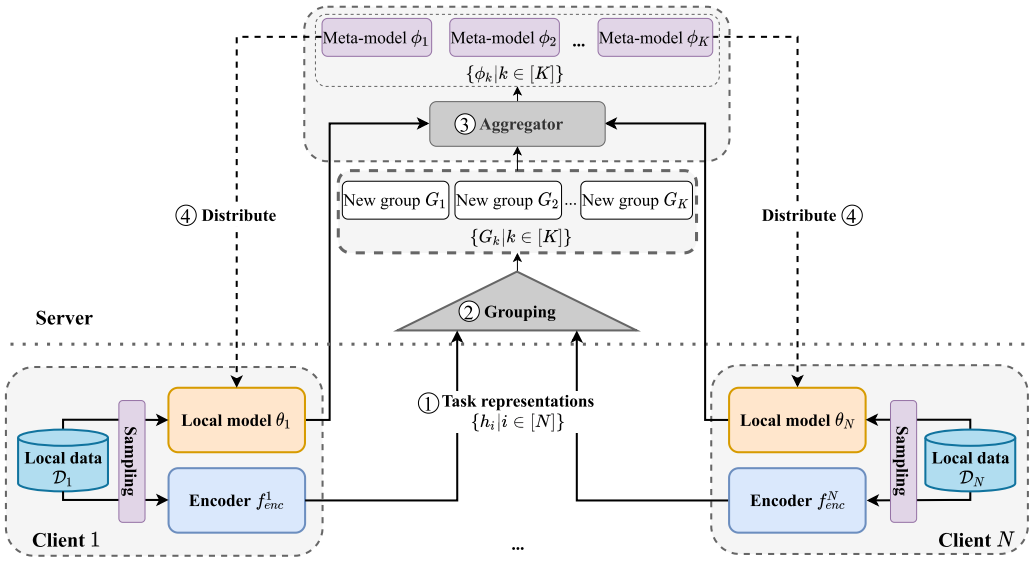

Fig. 4. Overview of the **Federated Optimization** stage. On the client-side, each client uses the pre-trained encoder f_{enc} to compress the training data into a latent representation h. On the server-side, the server generates new groups by performing grouping based on the representations of the clients. And then, each group updates the group-wise meta-model using the client model θ. Finally, the server distributes the group-wise meta-models to corresponding clients for the next round.

batch of training data $\mathcal{D}_i^{train',t}$:

$$\begin{aligned} \theta_i^{t+1} &= \theta_i^t - \alpha \nabla_{\theta_i^t} \mathcal{L}\left(\mathcal{D}_i^{train',t}\right). \\ \text{s.t. } \theta_i^t &= \phi_k^t \end{aligned} \tag{6}$$

After the client finishes updating the model θ_i^t, the client needs to use the updated model θ_i^{t+1} to evaluate and update the group-wise meta-model $\phi_{k,i}^t$, using a new batch of training data $\mathcal{D}_i^{train'',t}$:

$$\phi_{k,i}^{t+1} = \phi_{k,i}^t - \beta \nabla_{\phi_{k,i}^t} \mathcal{L}\left(\theta_i^{t+1}; \mathcal{D}_i^{train'',t}\right), \tag{7}$$

where β is the meta-learning rate and $\phi_{k,i}^{t+1}$ will be used in the next iteration at client i.

In each epoch $t \in T$, the sampled data $\mathcal{D}_i^{train',t}$ and $\mathcal{D}_i^{train'',t}$ consist of a meta task. We ues the last meta task to achieve the task representation h_i, as follows:

$$h_i = \frac{1}{\left|\left\{\mathcal{D}_i^{train',T}, \mathcal{D}_i^{train'',T}\right\}\right|} \sum_{x \in \{\mathcal{D}_i^{train',T}, \mathcal{D}_i^{train'',T}\}} f_{enc}^i(x), \tag{8}$$

Grouping step: As all clients complete local updates, clients transmit the updated group-wise meta-model $\phi_{k,i}$ and the task representation h_i to the server. For grouping clients based on similarity, a heuristic way is to measure the distance between the task representation h_i and the cluster center C_k. Here, we use the cosine similarity $cos(h_i, C_k)$ to define the distance between the representation of h_i and C_k:

$$cos(h_i, C_k) = \frac{h_i \cdot C_k}{\|h_i\| \, \|C_k\|}. \tag{9}$$

ALGORITHM 2: The training process for updating the group-wise meta-models

Input: Total communication round R, number of selected clients per round $|\mathcal{S}|$, number of local epochs T, training data $\mathcal{D}_i^{train}$ of client, learning rate α, meta-learning rate β, batch size $\mathcal{B}$, list of the encoder $\{f_{enc}^0, \ldots, f_{enc}^N\}$, initial group-wise meta-model $\{\phi_1^0, \ldots, \phi_K^0\}$, initial grouping result $G_1^0, \ldots, G_K^0$.

Output: List of updated group-wise meta-models $\{\phi_1^R, \ldots, \phi_K^R\}$

for $r = 1, \ldots, R$ **do**
  Server randomly selectes a subset of clients $\mathcal{S}$ with size Q;
  Server sends ϕ_k^r to all clients in $\mathcal{S}$ based on the grouping result $G_1^r, \ldots, G_K^r$;
  for each client i in $\mathcal{S}$ **in parallel do**
    $\phi_{k,i}^{r+1}, h_i$ = **ClientUpdate(**i, ϕ_k^r**)**;
    Client i sends $(\phi_{k,i}^{r+1}, h_i)$ back to the server;
  end
  Achieve the grouping result $\{G_1^{r+1}, \ldots, G_K^{r+1}\}$ using the task representation h_i based on Equation (10);
  Server performs a aggregation for each group: $\phi_k^{r+1} = \sum_{i \in G_k} w_i \phi_{k,i}^{r+1}, \forall k \in K$;
end

Function ClientUpdate(i, ϕ_k^r)**:**
  for $t = 1, \ldots, T$ **do**
    Randomly sampled data $(\mathcal{D}_i^{train',t}, \mathcal{D}_i^{train'',t})$ from $\mathcal{D}_i^{train}$ with batch size $\mathcal{B}$;
    Set $\theta_i^{t+1} = \phi_{k,i}^{r,t} - \alpha \nabla_{\phi_{k,i}^{r,t}} \mathcal{L}(\mathcal{D}_i^{train',t})$;
    Set $\phi_{k,i}^{r,t+1} = \phi_{k,i}^{r,t} - \beta \nabla_{\phi_{k,i}^{r,t}} \mathcal{L}(\theta_i^{t+1}; \mathcal{D}_i^{train'',t})$;
  end
  Learn the task representation h_i based on Equation (8);
  return $(\phi_{k,i}^{r,T}, h_i)$;
End Function

Hence, the client i estimates its group identify via finding the cluster center with highest similarity:

$$g_i = \arg\max_k cos(h_i, C_k), \tag{10}$$

where g_i is the group identify of client i.

Aggregation step: After the grouping step, the server achieves the grouping result $G_1, \ldots, G_K$. Then, for every round $r \in R$. Each group performs a aggregation by weighted averaging to achieve the aggregated group-wise meta-model ϕ_k:

$$\phi_k^{r+1} = \sum_{i \in G_k^r} w_i \phi_{k,i}^r, \tag{11}$$

where $w_i = \frac{|\mathcal{D}_i^{train}|}{\sum_{j \in \mathcal{S}} \mathcal{D}_j^{train}}$ and the weight w_i of each client i depends on the size of its local data $\mathcal{D}_i^{train}$.

4.4 The Personalization Stage

Finally, we perform personalization for all clients using the updated group-wise meta-models in the **Personalization** stage. Algorithm 3 illustrates this stage in detail. Given the dataset $\mathcal{D}_i^{test}$

for personalization, client i learns the representation H_i of data $\mathcal{D}_i^{test}$ based on Equation (8) and then upload it to the server. The server returns the optimal group-wise meta-model ϕ_k to client using the representation H_i based on Equation (10). The client can quickly adapt the group-wise meta-model through one-step gradient descent and achieves the personalized model θ_i:

$$\theta_i = \phi_k - \alpha \nabla \mathcal{L}(\mathcal{D}_i^{test}), \tag{12}$$

ALGORITHM 3: The process of personalization

Input: Dataset for personalization $\mathcal{D}_i^{test}$, the encoder of client f_{enc}^i
Output: List of personalized models $\{\theta_1, \ldots, \theta_N\}$

```
for each client i in parallel do
    Learn the representation H_i of data D_i^test using f_enc^i based on 8;
    Client i sends H_i back to the server and then gets the group-wise meta-model φ_k from the server;
    Obtain the personalized model: θ_i = φ_k − α∇L(D_i^test);
end
```

4.5 Algorithm Analysis and Discussion

Our framework incurs additional overhead in the grouping phase compared with the existing federated meta-learning. The additional computational complexity of grouping is $O(K|\mathcal{S}|d_h)$, where K is the number of groups, $|\mathcal{S}|$ is the number of selected clients per round and d_h is the dimension of task representations. Luckily, d_h is usually a small value because the task representation is a low-dimensional representation generated by the encoder. In the practical FL applications, $|\mathcal{S}|$ is a fraction of clients for efficiency, and K is often significantly less than $|\mathcal{S}|$. Therefore, our algorithm can be efficient in practice. Furthermore, as we need to store K group-wise meta-models, thus the additional space complexity is $O(Kd_\theta)$, where d_θ is the number of model parameters. Due to K being a small value, our framework does not have a high additional memory cost at the server.

In the **Initialization** stage, the performance of representation depends on the model architecture of the autoencoder. For example, a recursive autoencoder can deal with the document data, and a convolutional autoencoder can deal with the image data. In this work, although we adopt the stacked autoencoder to represent the data, it still achieves superior performance over the baselines, as shown in Section 5. Usually, we need to train the autoencoder to convergence to obtain the best performance. However, training the autoencoder to full convergence generally leads to high computation costs and is hard to be achieved for clients with constrained computation capability. Experimental results show that even if the autoencoder does not converge to the optimal value, our framework still performs relatively well.

Model initialization is also an important factor that affects the performance of our algorithm. In the CFL algorithms like **Iterative Federated Cluster Algorithm (IFCA)** [28], it assumes that the K group-wise global models should have good initialization to guarantee the performance. Due to the high training cost, it is inefficient to retry various initial models in practical federated learning. However, G-FML does not have this problem because it leverages meta-learning, which can naturally look for a good initial model without being specified manually. The stopping criterion of Algorithm 2 is the maximum number of communication rounds. As a parameter of our algorithm, it affects the learning accuracy and computational cost. In most cases, our algorithm does not stop until enough communication rounds such that the models converge. Nevertheless, we can limit the number of communication rounds to better tradeoff between the learning accuracy and computational cost.

5 EXPERIMENTS AND RESULTS

5.1 Datasets and Models

We conduct extensive experiments on benchmarking datasets from the FL benchmark—LEAF [44]—to evaluate the effectiveness and efficiency of our framework. These datasets include three Synthetic Datasets and two real-world datasets: FEMNIST [45] and Shakespeare [1]. Note that in our experiments, the FEMNIST dataset is generated based on the setup provided in [46]. The detailed descriptions of the datasets and models are listed as follows.

- **Synthetic Datasets:** We generate three synthetic datasets containing 10 classes and the feature dimension set as 60. We use 100 clients to train the classifiers collaboratively. More concretely, for each client $T \geq 1$, we generate data (x_t, y_t) consisting of features and labels, and their data distributions present a cluster structure with the number of clusters by the input hyperparameter M. Moreover, the probability of a cluster being selected satisfies the distribution $\sum_{j=1}^{M} p_j = 1$, and for all i and j, $p_i = p_j$. The cluster centers $\mu_j \in \mathbb{R}^s, j \in 1, \ldots, M$ are generated from $\mu_j \sim N(B_j, I), B_j \sim N(0, I)$. In order to generate local data for tth client, we select a cluster center μ_j according to the probability distribution $(p_1, \ldots, p_M)$, and the number of data samples n_t satisfies $n_t = \min(m_t + 50, 5000)$, where m_t is drawn from a log-normal distribution with mean 3 and sigma 2. The feature of a data sample is generated from $x_t^i \sim N(v_t, \sigma), v_t \sim N(C_t, I), C_t \sim N(0, I), \sigma_{i,i} = i^{-1.2}$, and its label satisfies $y_t^i = \arg\max(\text{sigmoid}(w_t x_t^i + N(0, 0.1 \cdot I))), w_t = Qu_t, u_t \sim N(\mu_t, I), w_t \in \mathbb{R}^{d+1}, Q \in \mathbb{R}^{d+1 \times s}$. Therefore, as M becomes higher, the data become more heterogeneous. In particular, we set $M = 1, 4, 8$ to generate these three synthetic datasets, respectively. All synthetic datasets are split randomly with 60% for training and 40% for testing. We use a multinomial logistic regression model. The input of this model is a flattened 60-dimensional vector, and the output is a class label between 0 and 9.
- **FEMNIST:** FEMNIST is a 62-class handwritten digits and characters image classification task, which is built by resampling the EMNIST [45]. First, we import the standard FEMNIST dataset of LEAF and generate a federated dataset of 200 clients according to train and test data distributions defined in [46]. In particular, there are ten classes ('a' - 'j') from EMNIST, and each client holds five of the 10 classes. The dataset is split randomly with 90% for training and 10% for testing. We use a two-layer DNN with a hidden layer of size 100. The input of this model is a flattened 784-dimensional vector, and the output is a class label between 0 and 9.
- **Shakespeare:** Shakespeare is a language modeling dataset, which is derived from "The Complete Works of William Shakespeare" [1]. Each speaking role in the play is considered a different client. We subsample 103 speaking roles to train a deep language model for next-character prediction. The dataset is split randomly with 70% for training and 30% for testing. We use a two-layer LSTM classifier containing 100 hidden units with an 8-dimensional embedding layer. The input of this model is a sequence of 80 characters, and the output is a character per training sample following an LSTM layer and a fully connected layer.

Table 2 reports the statistics of these datasets.

5.2 Compared Baselines

To demonstrate the effectiveness of *G-FML* framework, we compare the following approaches with our framework:

- **FedAvg** [1]**:** FedAvg is a vanilla federated learning approach. Each client trains the model locally and sends the model updates to a server, where the server aggregates the model

Table 2. The Statistics of the Datasets in Our Experiments

Dataset	Devices	Samples	Samples per device	
			mean	stdev
Synthetic (1)	100	36, 425	364.25	798.31
Synthetic (4)	100	27, 557	275.57	612.30
Synthetic (8)	100	48, 210	482.10	1, 047.33
FEMNIST	200	18, 345	91.72	159.29
Shakespeare	103	42, 485	412.48	201.17

updates with weighted averaging. The global model serves as the final model for all clients.

- **FedAvg+Update:** We implement a personalization version of FedAvg, named FedAvg+Update. Unlike the vanilla FedAvg, FedAvg+Update uses the global model to fine-tune the local model, resulting in a personalized model for the client.
- **FedProx** [46]**:** FedProx is an optimization framework that tackles statistical heterogeneity in federated settings with convergence guarantees and improved robustness and stability. FedProx encompasses FedAvg as a special case. In all experiments, we set the scale of the proximal term to 0.001.
- **IFCA** [28]**:** IFCA is a framework that alternately tags clients to a group and optimizes the model parameters for the group. In this framework, clients in the same group share a global group model.
- **Per-FedAvg** [6]**:** Per-FedAvg is the state-of-the-art federated meta-learning algorithm. Each client trains the meta-model locally, and then the server receives and aggregates them. Clients obtain the personalized models by performing a few gradient descent steps upon the global meta-model.

5.3 Implementation Details

All code is implemented in PyTorch [47] version 1.6.0. We conduct our experiments on a machine with an AMD® Ryzen® Threadripper 3990x 64-core CPU and a Nvidia® 1080 Ti GPU. To make a fair comparison between *G-FML* and other baselines, we use one batch of data at each epoch to train the local model. Each approach uses SGD as its local solver. Each client has a local testing set, and we evaluate the test accuracy of its corresponding personalized model based on these data. Unless stated otherwise, we report the test accuracy is defined by averaging over the test accuracy of all clients $\frac{1}{N}\sum_i Acc(\theta_i, \mathcal{D}_i^{test})$.

For all experiments, we use a fixed size subset of clients $|\mathcal{S}| = 10$. For all synthetic data experiments, the learning rate is 0.01. For FEMNIST and Shakespeare datasets, we use the learning rate of 0.003 and 0.8. In **Initialization** stage, each client trains the local autoencoder with 50 iterations. Both the encoder and the decoder are the multi-layer perceptron network composed of two fully connected layers for the autoencoder. Specifically, for Synthetic, FEMNIST, and Shakespeare datasets, the encoder section has the hidden layers of size (128, 6), (128, 64), and (128, 8), respectively. In **Personalization** stage, each client performs one-step gradient descent over its testing dataset to achieve the personalized model.

5.4 Effects of Non-IID Data

To study how Non-IID data affects convergence, we generated datasets with different levels of statistical heterogeneity using the synthetic method described in Section 5.1. Synthetic (1) denotes

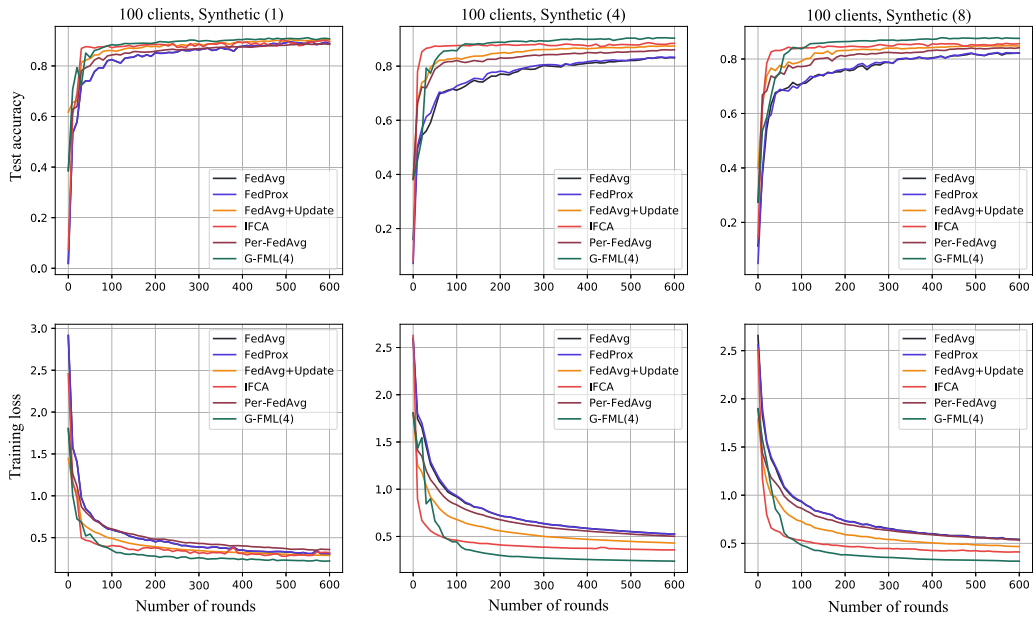

Fig. 5. Performance comparisons of baselines and *G-FML* (4) using different synthetic data (best viewed in color). It shows the test accuracy and training loss on three synthetic datasets whose statistical heterogeneity increases from left to right. *G-FML* improves the test accuracy on all synthetic datasets.

the true models of clients are clustered in one center. Thus, the data becomes more heterogeneous as the number of cluster centers increases. Specifically, we generate three datasets with different Non-IID levels, including Synthetics (1), (4), and (8). For all synthetic data experiments, each client performs $T = 5$ local epochs with batch size $\mathcal{B} = 10$ and the meta-learning rate $\beta = 0.005$. Moreover, we fix $K = 4$ for *G-FML* and denote it as *G-FML* (4).

In Figure 5, we plot the average test accuracy and training loss of clients in 600 communication rounds. We see that the test accuracies on the Synthetic (1) dataset are similar for all approaches when the training process converges. However, as shown in Table 3, with the statistical heterogeneity increases, test accuracy becomes worse for *G-FML* with $K = 4$. Though *G-FML* performs worse, it achieves higher test accuracy. On the Synthetic (4) dataset and Synthetic (8) dataset, we observe that *G-FML* converges slower than IFCA before 120 communication rounds. After that, *G-FML* keeps on increasing and achieves higher accuracy, while IFCA has converged on a lower accuracy. Unexpectedly, FedAvg+Update achieves higher accuracy than Per-FedAvg, which may be because we use the first-order approximation version of Per-FedAvg to update, which ignores the second-order term. Overall, the empirical results illustrate that *G-FML* can handle different levels of Non-IID data, and the training process is more efficient.

5.5 Effects of K

The number of groups K is the key parameter of *G-FML* that affects model performance. For employing *G-FML* in practice, K should carefully tune it to fit different levels of Non-IID data. It is noted that when $K = 1$, *G-FML* is equivalent to Per-FedAvg. However, a larger K may be able to fit more complex data distributions. Still, it also means that *G-FML* needs to optimize more meta-models, which may require more rounds of communication to achieve convergence, thus incurring higher communication costs as well as computation costs. Smaller K does the opposite. To fairly

Table 3. Test Accuracy on Different Levels of Non-IID Synthetic Datasets

Methods	Synthetic (1)	Synthetic (4)	Synthetic (8)
FedAvg	89.62%	83.22%	82.21%
FedProx	89.59%	83.36%	82.38%
FedAvg+Update	90.50%	87.49%	84.90%
IFCA	90.54%	88.66%	85.74%
Per-FedAvg	88.68%	86.04%	84.15%
G-FML (Ours)	**91.17%**	**90.51%**	**87.84%**

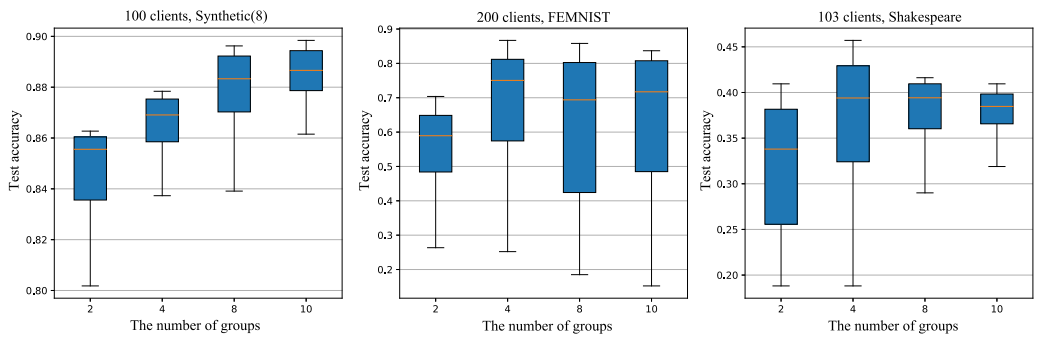

Fig. 6. *G-FML* with different values of K using Synthetic (4), FEMNIST, and Shakespeare datasets. It shows the boxplots of test accuracy on three heterogeneous datasets. Combined with Tables 3 and 4, *G-FML* achieves the best performance on most datasets with $K = 4$.

compare the performance of G-FML with different K values, we keep all hyperparameters the same except K. We monitor the performance of *G-FML* using several K.

We conduct experiments on Synthetic (8), FEMNIST, and Shakespeare datasets. K is from the limited candidate set $\{2, 4, 8, 10\}$. To take a closer look at the variations, we adopt a boxplot to depict the accuracies of different parameters K graphically. As shown in Figure 6, we observe that there exists an optimal K value for various datasets. Specifically, for the Synthetic (8) dataset, since we create this dataset with eight cluster centers, it performs the best with $K \geq 8$. For the FEMNIST dataset, although the performance of *G-FML* with $K = 4$ is similar to $K = 8$, $K = 4$ performs more stable during the training process. As for the Shakespeare dataset, $K = 4$ achieves the best performance, while when $K = 8$ and $K = 10$, *G-FML* fluctuates within a particular accuracy range. Therefore, K should be tuned carefully depending on the level of the heterogeneous dataset. In practice, we can run *G-FML* with different K values in a small number of communication rounds and then choose the value of K with the best performance. For example, For each value of K, we perform r rounds of communication and record the model performance at different K. What's more, we can use the expert's prior knowledge to determine a suitable K.

5.6 The Effectiveness of Our Grouping Strategy

To evaluate the effectiveness of our grouping strategy, we further compare our proposed approach with the other two grouping strategies. The comparison strategies are **Random Client Assignment** (**RCA**) and **Fixed Client Assignment** (**FCA**). More concretely, RCA randomly assigns

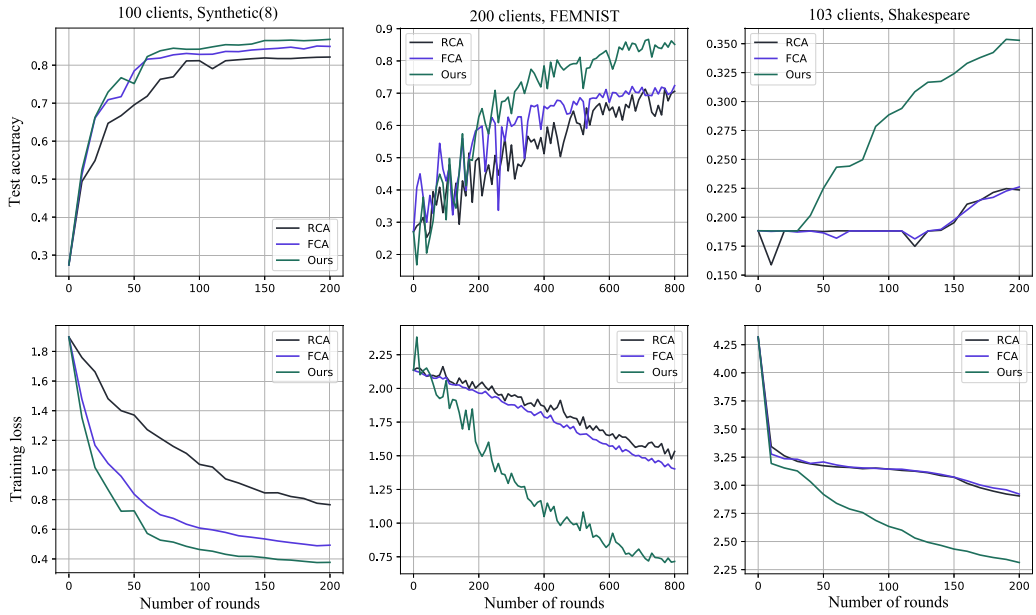

Fig. 7. *G-FML* with different grouping strategies (*RCA, FCA, and Ours*) on three heterogeneous datasets. The results demonstrate that our grouping strategy can effectively improve the model performance.

clients into groups in each communication round, and FCA uses the grouping result derived from the **Initialization** stage.

As shown in Figure 7, our dynamic grouping strategy is superior to RCA and FCA in model accuracy and convergence speed. Especially on the Shakespeare dataset, both RCA and FCA hardly converge to a target accuracy due to the high heterogeneity of the data. Nevertheless, our group strategy still keeps on a faster convergence speed. RCA performs worst because it only randomly assigns clients to groups ignoring the similarity of data distribution between clients. In comparison, FCA performs better than RCA by combining the similarity of data distribution between clients. As mentioned in Section 1, the information on data distribution provided by the clients is different in each round. Our grouping strategy leverages this information to adaptively divide the clients into groups in each communication round.

To further illustrate the effectiveness of our grouping strategy, we utilize the hierarchy of CIFIA-100 to partition clients into 20 groups where each group belongs to a superclass. Specifically, 100 clients participate in the training process, and each superclass is assigned to five clients. Then the task representations are reduced to two-dimension representations by T-SNE [48] and plotted in Figure 8. The learned task representations present a clear cluster structure.

5.7 Performance Comparisons

To highlight the empirical performance of *G-FML*, we perform comparisons between *G-FML* and the other baselines on three heterogeneous datasets. We fix the number of groups $K = 4$ for all experiments. We also set the randomly selected clients, the initial model parameters, and the personalized learning rate across all runs for a fair comparison.

In Figure 9, we plot the test accuracy and training loss in each communication round. We see that on all datasets, Per-FedAvg achieves worse convergence related to *G-FML* because of the

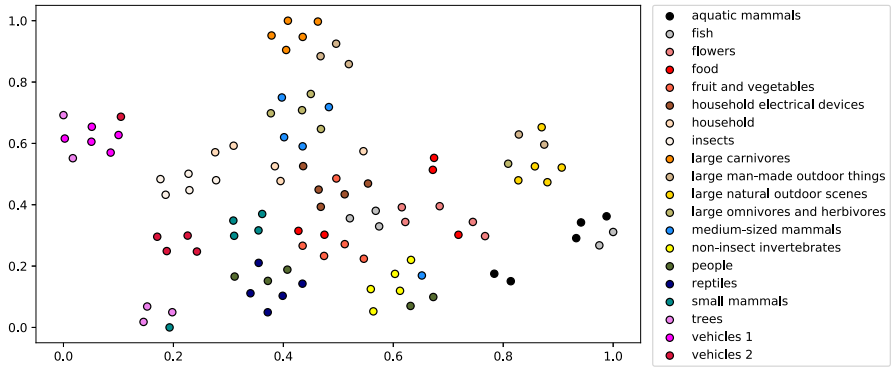

Fig. 8. T-SNE visualization of the task representations H for CIFIA-100 dataset. Marker color indicates different superclasses. We can observe that *G-FML* groups together clients from the same superclass.

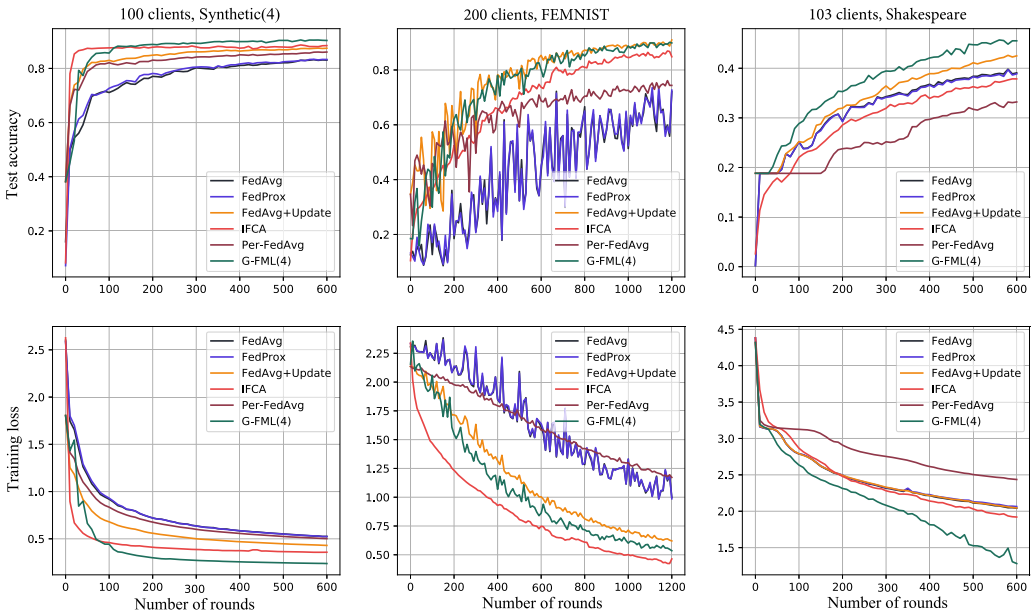

Fig. 9. Performance comparisons of baselines and *G-FML* on various datasets (best viewed in color).

highly skewed Non-IID data. Especially in the Shakespeare dataset, its convergence speed and the model accuracy are even worse than vanilla FedAvg. Furthermore, the experimental results demonstrate that *G-FML* not only results in more stable performance than the other baselines but also arrives at desired accuracy with fewer communication rounds. Table 4 presents the test accuracy on testing datasets. Due to the greater fluctuations in model accuracy caused by Non-IID data, we assume the early stopping strategy is applied. We regard the maximum test accuracy as the final model accuracy during the testing process. We observe an average accuracy increase of 3.55% in Synthetic, 13.98% in FEMNIST, and 12.50% in Shakespeare for *G-FML* compared to Per-FedAvg.

Table 4. Comparisons on Different Datasets

Methods	Synthetic (1)	Synthetic (4)	Synthetic (8)	FEMNIST	Shakespeare
FedAvg	89.62%	83.22%	82.21%	72.19%	39.71%
FedProx	89.59%	83.36%	82.38%	73.96%	39.81%
FedAvg+Update	90.50%	87.49%	84.90%	**90.96**%	42.61%
IFCA	90.54%	88.66%	85.74%	86.95%	37.86%
Per-FedAvg	88.68%	86.04%	84.15%	76.14%	33.21%
G-FML (Ours)	**91.17%**	**90.51%**	**87.84%**	90.12%	**45.71%**

6 CONCLUSION

In this work, we propose *G-FML*, an optimization framework that mitigates the inefficient training issue of the existing federated meta-learning framework on highly skewed Non-IID data. *G-FML* adaptively divides the clients with similar data distribution into the same group and applies meta-learning within each group to learn personalized models for the clients. Furthermore, we develop an efficient grouping algorithm for client grouping. Although *G-FML* needs a pre-processing stage to train an autoencoder, it arrives at desired accuracy with fewer communication rounds compared with vanilla FedAvg, IFCA, and Per-FedAvg. We also empirically show that *G-FML* can achieve significant improvement in the accuracy of overall heterogeneous datasets. In future work, the question of generalization to new clients with new data distribution awaits further analysis. Specifically, we plan to apply lifelong learning to our framework, using prior knowledge and quickly adapting to new data distributions.

REFERENCES

[1] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*. 1273–1282.

[2] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. 2018. Federated learning with non-IID data. arXiv:1806.00582. Retrieved from https://arxiv.org/abs/1806.00582.

[3] Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. 2020. On the convergence of FedAvg on non-IID data. In *Proceedings of the 8th International Conference on Learning Representations*.

[4] Chuhan Wu, Fangzhao Wu, Yang Cao, Yongfeng Huang, and Xing Xie. 2021. FedGNN: Federated graph neural network for privacy-preserving recommendation. arXiv:2102.04925. Retrieved from https://arxiv.org/abs/2102.04925.

[5] Virginia Smith, Chao-Kai Chiang, Maziar Sanjabi, and Ameet S. Talwalkar. 2017. Federated multi-task learning. In *Proceedings of the Advances in Neural Information Processing Systems*. 4424–4434.

[6] Alireza Fallah, Aryan Mokhtari, and Asuman E. Ozdaglar. 2020. Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach. In *Proceedings of the Advances in Neural Information Processing Systems*. 3557–3568.

[7] Yuyang Deng, Mohammad Mahdi Kamani, and Mehrdad Mahdavi. 2020. Adaptive personalized federated learning. arXiv:2003.13461. Retrieved from https://arxiv.org/abs/2003.13461.

[8] Canh T. Dinh, Nguyen H. Tran, and Tuan Dung Nguyen. 2020. Personalized federated learning with moreau envelopes. In *Proceedings of the Advances in Neural Information Processing Systems*. 21394–21405.

[9] Yutao Huang, Lingyang Chu, Zirui Zhou, Lanjun Wang, Jiangchuan Liu, Jian Pei, and Yong Zhang. 2021. Personalized cross-silo federated learning on non-IID data. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 7865–7873.

[10] Fei Chen, Zhenhua Dong, Zhenguo Li, and Xiuqiang He. 2018. Federated meta-learning for recommendation. arXiv:1802.07876. Retrieved from https://arxiv.org/abs/1802.07876.

[11] Yihan Jiang, Jakub Konečný, Keith Rush, and Sreeram Kannan. 2019. Improving federated learning personalization via model agnostic meta learning. arXiv:1909.12488. Retrieved from https://arxiv.org/abs/1909.12488.

[12] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning*. JMLR.org, 1126–1135.

[13] Alex Nichol, Joshua Achiam, and John Schulman. 2018. On first-order meta-learning algorithms. arXiv:1803.02999. Retrieved from https://arxiv.org/abs/1803.02999.
[14] Manoj Ghuhan Arivazhagan, Vinay Aggarwal, Aaditya Kumar Singh, and Sunav Choudhary. 2019. Federated learning with personalization layers. arXiv:1912.00818. Retrieved from https://arxiv.org/abs/1912.00818.
[15] Aviv Shamsian, Aviv Navon, Ethan Fetaya, and Gal Chechik. 2021. Personalized federated learning using hypernetworks. In *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 9489–9502.
[16] Liam Collins, Hamed Hassani, Aryan Mokhtari, and Sanjay Shakkottai. 2021. Exploiting shared representations for personalized federated learning. In *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 2089–2099.
[17] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong. 2019. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology* 10, 2 (2019), 1–19.
[18] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. 2020. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine* 37, 3 (2020), 50–60.
[19] Peter Kairouz, H. Brendan McMahan, Brendan Avent, et al. 2021. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning* 14, 1–2 (2021), 1–210.
[20] Viraaji Mothukuri, Reza M. Parizi, Seyedamin Pouriyeh, Yan Huang, Ali Dehghantanha, and Gautam Srivastava. 2021. A survey on security and privacy of federated learning. *Future Generation Computer Systems* 115 (2021), 619–640.
[21] Viraj Kulkarni, Milind Kulkarni, and Aniruddha Pant. 2020. Survey of personalization techniques for federated learning. In *Proceedings of the 2020 4th World Conference on Smart Trends in Systems, Security, and Sustainability*. IEEE, 794–797.
[22] Alysa Ziying Tan, Han Yu, Lizhen Cui, and Qiang Yang. 2022. Towards personalized federated learning. *IEEE Transactions on Neural Networks and Learning Systems* (2022).
[23] Eunjeong Jeong, Seungeun Oh, Hyesung Kim, Jihong Park, Mehdi Bennis, and Seong-Lyun Kim. 2018. Communication-efficient on-device machine learning: Federated distillation and augmentation under non-IID private data. arXiv:1811.11479. Retrieved from https://arxiv.org/abs/1811.11479.
[24] Moming Duan, Duo Liu, Xianzhang Chen, Renping Liu, Yujuan Tan, and Liang Liang. 2021. Self-balancing federated learning with global imbalanced data in mobile systems. *IEEE Transactions on Parallel and Distributed Systems* 32, 1 (2021), 59–71.
[25] Hao Wang, Zakhary Kaplan, Di Niu, and Baochun Li. 2020. Optimizing federated learning on non-IID data with reinforcement learning. In *Proceedings of the IEEE INFOCOM*. IEEE, 1698–1707.
[26] Miao Yang, Ximin Wang, Hongbin Zhu, Haifeng Wang, and Hua Qian. 2021. Federated learning with class imbalance reduction. In *Proceedings of the 29th European Signal Processing Conference*. IEEE, 2174–2178.
[27] Felix Sattler, Klaus-Robert Müller, and Wojciech Samek. 2021. Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints. *IEEE Transactions on Neural Networks and Learning Systems* 32, 8 (2021), 3710–3722.
[28] Avishek Ghosh, Jichan Chung, Dong Yin, and Kannan Ramchandran. 2020. An efficient framework for clustered federated learning. In *Proceedings of the Advances in Neural Information Processing Systems*. 19586–19597.
[29] Christopher Briggs, Zhong Fan, and Peter Andras. 2020. Federated learning with hierarchical clustering of local updates to improve training on non-IID data. In *Proceedings of the 2020 International Joint Conference on Neural Networks*. IEEE, 1–9.
[30] Moming Duan, Duo Liu, Xinyuan Ji, Renping Liu, Liang Liang, Xianzhang Chen, and Yujuan Tan. 2021. FedGroup: Efficient federated learning via decomposed similarity-based clustering. In *Proceedings of the IEEE International Conference on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking*. 228–237.
[31] Daliang Li and Junpu Wang. 2019. FedMD: Heterogenous federated learning via model distillation. arXiv:1910.03581. Retrieved from https://arxiv.org/abs/1910.03581.
[32] Yiqiang Chen, Xin Qin, Jindong Wang, Chaohui Yu, and Wen Gao. 2020. FedHealth: A federated transfer learning framework for wearable healthcare. *IEEE Intelligent Systems* 35, 4 (2020), 83–93.
[33] Ming Xie, Guodong Long, Tao Shen, Tianyi Zhou, Xianzhi Wang, Jing Jiang, and Chengqi Zhang. 2021. Multi-center federated learning. arXiv:2108.08647. Retrieved from https://arxiv.org/abs/2108.08647.
[34] Duc Bui, Kshitiz Malik, Jack Goetz, Honglei Liu, Seungwhan Moon, Anuj Kumar, and Kang G. Shin. 2019. Federated user representation learning. arXiv:1909.12535. Retrieved from https://arxiv.org/abs/1909.12535.
[35] Paul Pu Liang, Terrance Liu, Ziyin Liu, Ruslan Salakhutdinov, and Louis-Philippe Morency. 2020. Think locally, act globally: Federated learning with local and global representations. arXiv:2001.01523. Retrieved from https://arxiv.org/abs/2001.01523.
[36] Timothy M. Hospedales, Antreas Antoniou, Paul Micaelli, and Amos J. Storkey. 2020. Meta-learning in neural networks: A survey. arXiv:2004.05439. Retrieved from https://arxiv.org/abs/2004.05439.

[37] Jiang Lu, Pinghua Gong, Jieping Ye, and Changshui Zhang. 2020. Learning from very few samples: A survey. arXiv:2009.02653. Retrieved from https://arxiv.org/abs/2009.02653.
[38] Sen Lin, Guang Yang, and Junshan Zhang. 2020. A collaborative learning framework via federated meta-learning. In *Proceedings of the 40th IEEE International Conference on Distributed Computing Systems*. IEEE, 289–299.
[39] Mikhail Khodak, Maria-Florina Balcan, and Ameet S. Talwalkar. 2019. Adaptive gradient-based meta-learning methods. In *Proceedings of the Advances in Neural Information Processing Systems*. 5915–5926.
[40] Risto Vuorio, Shao-Hua Sun, Hexiang Hu, and Joseph J. Lim. 2019. Multimodal model-agnostic meta-learning via task-aware modulation. In *Proceedings of the Advances in Neural Information Processing Systems*. 1–12.
[41] Huaxiu Yao, Ying Wei, Junzhou Huang, and Zhenhui Li. 2019. Hierarchically structured meta-learning. In *Proceedings of the 36th International Conference on Machine Learning*. PMLR, 7045–7054.
[42] Quoc V. Le and Tomás Mikolov. 2014. Distributed representations of sentences and documents. In *Proceedings of the 31th International Conference on Machine Learning*. JMLR.org, 1188–1196.
[43] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Association for Computational Linguistics, 3980–3990.
[44] Sebastian Caldas, Peter Wu, Tian Li, Jakub Konečný, H. Brendan McMahan, Virginia Smith, and Ameet Talwalkar. 2018. LEAF: A benchmark for federated settings. arXiv:1812.01097. Retrieved from https://arxiv.org/abs/1812.01097.
[45] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and André van Schaik. 2017. EMNIST: Extending MNIST to handwritten letters. In *Proceedings of the 2017 International Joint Conference on Neural Networks*. IEEE, 2921–2926.
[46] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated optimization in heterogeneous networks. In *Proceedings of the Machine Learning and Systems*. mlsys.org.
[47] Adam Paszke, Sam Gross, Francisco Massa, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Proceedings of the Advances in Neural Information Processing Systems*. 8024–8035.
[48] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of Machine Learning Research* 9, 11 (2008).

Received 3 November 2021; revised 22 June 2022; accepted 15 August 2022